Group 6 Project

Olivia Jones

```r
set.seed(445)
main_dir <- "sample_features_balanced.csv"

#list of subdirectories and files
subfolders <- list.dirs(main_dir, recursive = FALSE)

#empty list
audio_data <- list()

for (subfolder in subfolders) {
  label <- basename(subfolder)
  files <- list.files(subfolder, pattern = "\\.wav$", full.names = TRUE)
  for (file in files) {
    audio <- readWave(file)
    spec_data <- spectro(audio, f = audio@samp.rate, wl = 512, ovlp = 75, plot = FALSE)
    spectrogram_matrix <- as.data.frame(spec_data$amp)

    audio_data[[length(audio_data) + 1]] <- list(
      file = file,
      label = label,
      spectrogram = spectrogram_matrix
    )
  }
}
```

```r
#set.seed(445)
sound_data <- read.csv("sample_features_balanced.csv")
head(sound_data)
```

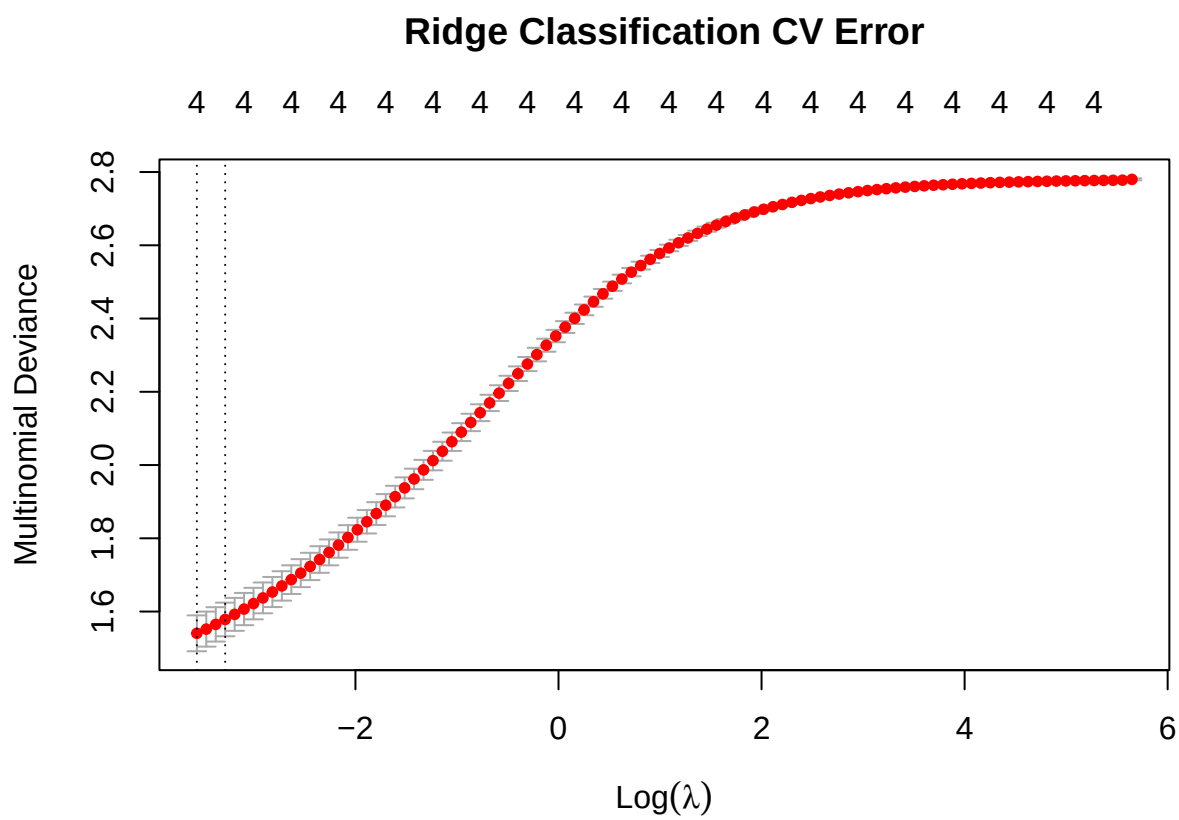
```
##   core_frequency max_amplitude average_activation   duration label
## 1      1063.1717  3.059000e+04        0.000640756 0.08730159   hat
## 2       539.3926  1.027420e+09        0.000792523 0.05210884   hat
## 3       540.5202  2.001861e+09        0.002079763 0.10000000   hat
## 4       545.7077  9.874376e+08        0.003245642 0.75523810   hat
## 5      1963.2908  3.643806e-01        0.000855111 0.07061225   hat
## 6       545.5926  1.771449e+09        0.003318707 0.75523810   hat
```

```r
summary(sound_data)
```

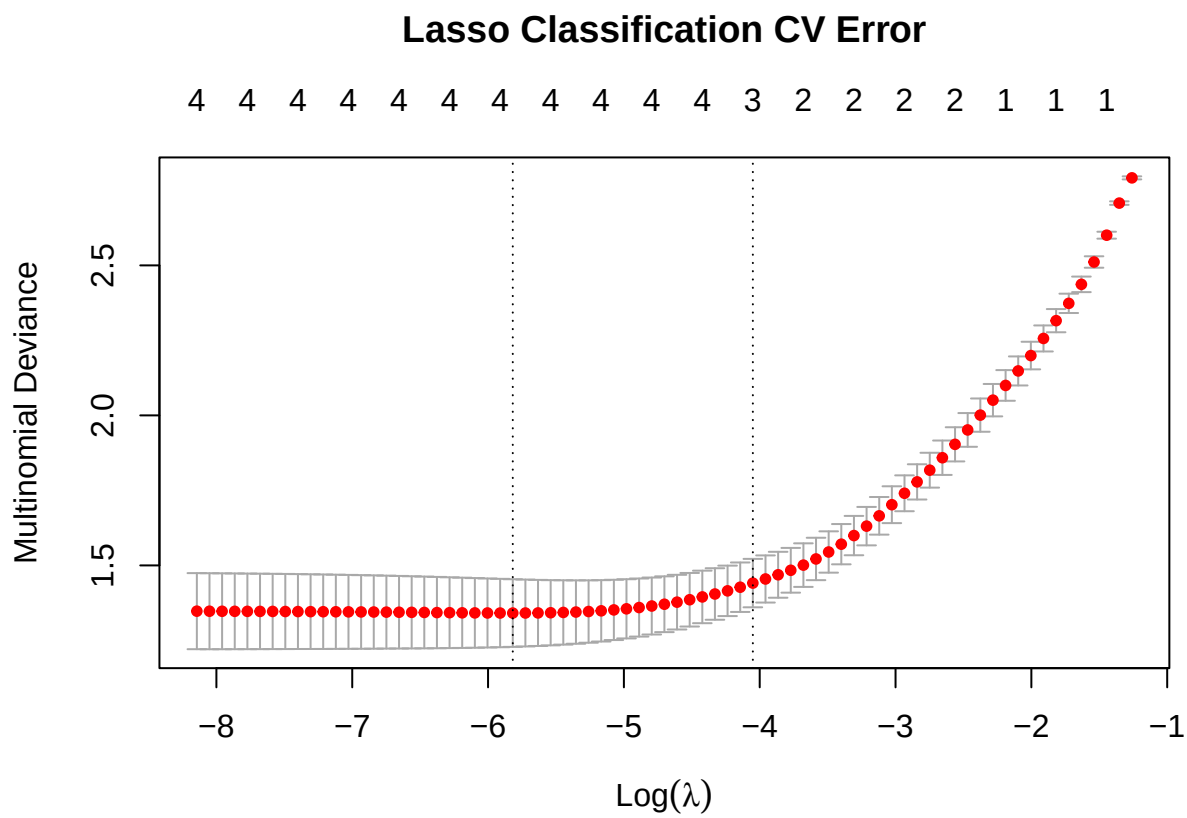
```
##  core_frequency    max_amplitude       average_activation     duration
##  Min.   :  32.39   Min.   :0.000e+00   Min.   :0.0003003   Min.   :0.03231
##  1st Qu.:  69.50   1st Qu.:2.843e+04   1st Qu.:0.0036056   1st Qu.:0.27357
##  Median : 267.25   Median :1.448e+09   Median :0.0067618   Median :0.67100
##  Mean   : 467.39   Mean   :1.140e+09   Mean   :0.0062646   Mean   :1.47459
##  3rd Qu.: 573.05   3rd Qu.:2.027e+09   3rd Qu.:0.0087507   3rd Qu.:1.81272
```

```
##  Max.   :1971.33   Max.   :2.147e+09   Max.   :0.0118515   Max.   :9.60000
##     label
##  Length:412
##  Class :character
##  Mode  :character
##
##
##
```

```r
set.seed(445)
audio_data <- read.csv("sample_features_balanced.csv")
# Separate predictors and response variable
X <- as.matrix(audio_data[ , -which(names(audio_data) == "label")]) # All columns except label
Y <- audio_data$label

# ridge regression (alpha = 0) for classification
cv_ridge <- cv.glmnet(X, Y, alpha = 0, family = "multinomial")
# lasso regression (alpha = 1) for classification
cv_lasso <- cv.glmnet(X, Y, alpha = 1, family = "multinomial") #more than two categories: multinomial

#cross-validation error for ridge and lasso classification
plot(cv_ridge)
title("Ridge Classification CV Error", line = 3)
```




```r
plot(cv_lasso)
title("Lasso Classification CV Error", line = 3)
```



- The graphs illustrate the cross-validation errors for Ridge and Lasso classification models across different values of the regularization parameter λ which is plotted on a logarithmic scale.
- For Ridge regression, the multinomial deviance (error), decreases initially as λ decreases, reaching a minimum value near $log(\lambda) = -2$. After this point, λ continues to decrease and the error starts increasing due to over-fitting. This behavior reflects the balance between bias and variance, where smaller λ values introduce higher variance in the model
- For Lasso regression, the multinomial deviance is minimized around $log(\lambda) = -6$. However, as λ decreases further, the error increases sharply, indicating that the model becomes too flexible and overfits the training data. The error for Lasso stays flat across a wide range of λ values before reaching the optimal λ indicates that Lasso is more robust to changes in λ compared to Ridge.
- The dotted vertical lines in both graphs indicate the optimal λ values chosen through cross-validation represents the point where the error is minimized. These graphs shows that the tuning of λ achieves the best balance between underfitting and overfitting for each regularization method.

```r
set.seed(445)
# training and testing sets
train_indices <- sample(1:nrow(audio_data), size = 0.8 * nrow(audio_data))  # 80% training data
train_data <- audio_data[train_indices, ]
test_data <- audio_data[-train_indices, ]

# separate predictors and response for training and testing
X_train <- as.matrix(train_data[ , -which(names(train_data) == "label")])
Y_train <- as.factor(train_data$label)

X_test <- as.matrix(test_data[ , -which(names(test_data) == "label")])
Y_test <- as.factor(test_data$label)
```

```r
#Train
# Train Lasso regression (alpha = 1)
cv_lasso <- cv.glmnet(X_train, Y_train, alpha = 1, family = "multinomial")
# Train Ridge regression (alpha = 0)
cv_ridge <- cv.glmnet(X_train, Y_train, alpha = 0, family = "multinomial")



## Predict
# Predict class labels using Lasso
lasso_predictions <- predict(cv_lasso, X_test, type = "class", s = cv_lasso$lambda.min)
# Predict class labels using Ridge
ridge_predictions <- predict(cv_ridge, X_test, type = "class", s = cv_ridge$lambda.min)


print(cv_lasso$lambda.min)
```

```
## [1] 0.005166839
```

```r
print(cv_ridge$lambda.min)
```

```
## [1] 0.02822271
```

```r
# Predict class labels using Lasso
lasso_predictions <- predict(cv_lasso, X_test, type = "class", s = cv_lasso$lambda.min)
# Predict class labels using Ridge
ridge_predictions <- predict(cv_ridge, X_test, type = "class", s = cv_ridge$lambda.min)


# accuracy for Lasso
lasso_accuracy <- mean(lasso_predictions == Y_test)
print(paste("Lasso Accuracy:", round(lasso_accuracy * 100, 2), "%"))
```

```
## [1] "Lasso Accuracy: 75.9 %"
```

```r
# accuracy for Ridge
ridge_accuracy <- mean(ridge_predictions == Y_test)
print(paste("Ridge Accuracy:", round(ridge_accuracy * 100, 2), "%"))
```

```
## [1] "Ridge Accuracy: 78.31 %"
```

- This is an accuracy approach that is basic and focuses only on raw accuracy. It doesn't provide any insights into where the models fail like class-wise performance or misclassifications.
- The accuracy obtained here, 75.9% for Lasso and 78.31% for Ridge, is poor and doesn't grasp advanced tuning techniques or diagnostics.
- The values 0.005166839 and 0.02822271 correspond to the lambda values (regularization parameters) chosen during cross-validation for the Lasso and Ridge models.

```r
set.seed(445)
library(glmnet)
library(caret)
```

```
## Loading required package: lattice

##
## Attaching package: 'caret'

## The following object is masked from 'package:purrr':
```

```
##
##     lift
```

```r
# Split the data into training and testing sets using stratified sampling
set.seed(445)  # For reproducibility
train_indices <- createDataPartition(audio_data$label, p = 0.8, list = FALSE)
train_data <- audio_data[train_indices, ]
test_data <- audio_data[-train_indices, ]

# Separate predictors and response for training and testing
X_train <- as.matrix(train_data[, -which(names(train_data) == "label")])
Y_train <- as.factor(train_data$label)

X_test <- as.matrix(test_data[, -which(names(test_data) == "label")])
Y_test <- as.factor(test_data$label)

# Standardize the predictors
X_train_scaled <- scale(X_train)
X_test_scaled <- scale(X_test)

# Train Lasso regression (alpha = 1)
cv_lasso <- cv.glmnet(X_train_scaled, Y_train, alpha = 1, family = "multinomial")

# Train Ridge regression (alpha = 0)
cv_ridge <- cv.glmnet(X_train_scaled, Y_train, alpha = 0, family = "multinomial")

# Predict class labels using Lasso
lasso_predictions <- predict(cv_lasso, X_test_scaled, type = "class", s = cv_lasso$lambda.min)

# Predict class labels using Ridge
ridge_predictions <- predict(cv_ridge, X_test_scaled, type = "class", s = cv_ridge$lambda.min)

# Calculate accuracy for Lasso
lasso_accuracy <- mean(lasso_predictions == Y_test)
print(paste("Lasso Accuracy:", round(lasso_accuracy * 100, 2), "%"))
```

```
## [1] "Lasso Accuracy: 82.5 %"
```

```r
# Calculate accuracy for Ridge
ridge_accuracy <- mean(ridge_predictions == Y_test)
print(paste("Ridge Accuracy:", round(ridge_accuracy * 100, 2), "%"))
```

```
## [1] "Ridge Accuracy: 80 %"
```

```r
# Tune (Regularization Strength)
# Lasso
lasso_predictions_tuned <- predict(cv_lasso, X_test_scaled, type = "class", s = cv_lasso$lambda.min * 0
lasso_accuracy_tuned <- mean(lasso_predictions_tuned == Y_test)
print(paste("Tuned Lasso Accuracy:", round(lasso_accuracy_tuned * 100, 2), "%"))
```

```
## [1] "Tuned Lasso Accuracy: 82.5 %"
```

```r
# Ridge
ridge_predictions_tuned <- predict(cv_ridge, X_test_scaled, type = "class", s = cv_ridge$lambda.min * 0
ridge_accuracy_tuned <- mean(ridge_predictions_tuned == Y_test)
print(paste("Tuned Ridge Accuracy:", round(ridge_accuracy_tuned * 100, 2), "%"))
```

```
## [1] "Tuned Ridge Accuracy: 80 %"
```

```r
# Elastic Net (alpha = 0.5)
cv_elastic <- cv.glmnet(X_train_scaled, Y_train, alpha = 0.5, family = "multinomial")
elastic_predictions <- predict(cv_elastic, X_test_scaled, type = "class", s = cv_elastic$lambda.min)
elastic_accuracy <- mean(elastic_predictions == Y_test)
print(paste("Elastic Net Accuracy:", round(elastic_accuracy * 100, 2), "%"))
```

```
## [1] "Elastic Net Accuracy: 81.25 %"
```

- The accuracy metrics give an overview of how well each model performs in classifying the data. The Lasso regression model achieves an accuracy of 82.5%, which means that 82.5% of the test samples were correctly classified. Ridge regression follows with an accuracy of 80%, showing slightly lower performance compared to Lasso, which may be due to Ridge's inability to perform feature selection as effectively. After hyperparameter tuning, the accuracy for both Lasso and Ridge models remains unchanged, suggesting that the models were already performing optimally with the default parameters. The Elastic Net model, which combines Lasso and Ridge regularization effects, achieves an accuracy of 81.25%. This is slightly better than Ridge but not as strong as Lasso, meaning it's a balanced approach between the two methods.

```r
# confusion matrices
print("Lasso Confusion Matrix:")
```

```
## [1] "Lasso Confusion Matrix:"
```

```r
print(table(Lasso_Predicted = lasso_predictions, True = Y_test))
```

```
##                 True
## Lasso_Predicted hat kick ride snare
##           hat    18    0    3     2
##           kick    0   14    0     1
##           ride    1    2   17     0
##           snare   1    4    0    17
```

```r
print("Ridge Confusion Matrix:")
```

```
## [1] "Ridge Confusion Matrix:"
```

```r
print(table(Ridge_Predicted = ridge_predictions, True = Y_test))
```

```
##                 True
## Ridge_Predicted hat kick ride snare
##           hat    18    0    3     2
##           kick    0   15    1     2
##           ride    1    1   16     1
##           snare   1    4    0    15
```

```r
print("Elastic Net Confusion Matrix:")
```

```
## [1] "Elastic Net Confusion Matrix:"
```

```r
print(table(Elastic_Predicted = elastic_predictions, True = Y_test))
```

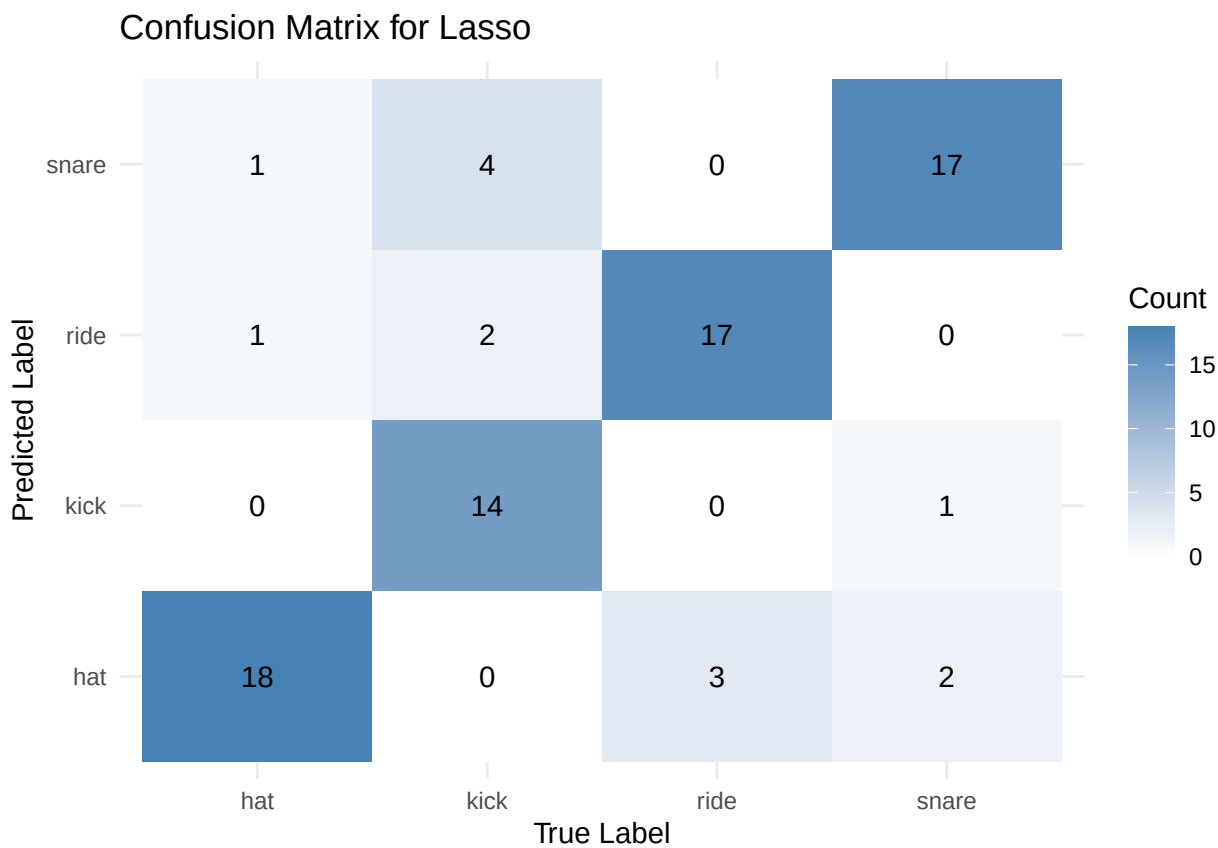
```
##                   True
## Elastic_Predicted hat kick ride snare
##             hat    18    0    3     2
##             kick    0   14    0     1
##             ride    1    2   17     1
##             snare   1    4    0    16
```

- The Lasso confusion matrix shows the model's ability to classify the test data into the four categories: hat, kick, ride, and snare. It shows that Lasso correctly classified 18 "hat," 14 "kick," 17 "ride," and 17 "snare" samples. However, there were some misclassifications, including three "ride" samples and two "snare" samples being incorrectly predicted as "hat." Although the model performs well overall, these misclassifications express some difficulty in distinguishing between certain categories.
- The Ridge confusion matrix shows the classification outcomes for the Ridge regression model. It correctly identifies 18 "hat," 15 "kick," 16 "ride," and 15 "snare" samples. However, it shows slightly higher misclassifications compared to Lasso. It struggles more with distinguishing between "ride" and "snare," which has more confusion between these categories. This could be due to Ridge's tendency to keep all features in the model, even the ones with weaker predictive power.
- The Elastic Net confusion matrix provides a balance between Lasso and Ridge models, with correct classifications of 18 "hat," 14 "kick," 17 "ride," and 16 "snare" samples. The misclassification patterns are similar to those in Lasso, but with slight differences in how the errors are distributed. Elastic Net achieves better balance by incorporating both Lasso's feature selection and Ridge's regularization, leading to moderate performance across all categories.

```r
#install.packages("reshape2")
library(reshape2)
```

```
##
## Attaching package: 'reshape2'

## The following object is masked from 'package:tidyr':
##
##     smiths
```

```r
set.seed(445)
library(ggplot2)

#confusion matrix for Lasso
lasso_cm <- table(Lasso_Predicted = lasso_predictions, True = Y_test)

# melt the confusion matrix
lasso_cm_melted <- melt(lasso_cm)

#  heatmap
ggplot(data = lasso_cm_melted, aes(x = True, y = Lasso_Predicted, fill = value)) +
  geom_tile() +
  scale_fill_gradient(low = "white", high = "steelblue") +
  geom_text(aes(label = value), color = "black") +
  labs(title = "Confusion Matrix for Lasso",
       x = "True Label",
       y = "Predicted Label",
       fill = "Count") +
  theme_minimal()
```



- The confusion heatmap for the Lasso model represents the model's classification performance by comparing the predicted labels to true labels, with darker blue cells as higher counts. The diagonal cells show correctly classified samples, with 18 "hat", 14 "kick", 17 "ride", and 17 "snare" samples accurately predicted. Off-diagonal cells are misclassifications, such as 3 "ride" samples misclassified as "hat", 2 "snare" samples misclassified as "hat", 1 "snare" misclassified as "kick," and 2 "ride" misclassified as "snare." Although the Lasso model shows strong overall performance, as indicated by most counts along the diagonal, misclassifications occur more frequently in the "snare" and "ride" categories, showing confusion with other classes. The "kick" category performs the best with the fewest misclassifications.

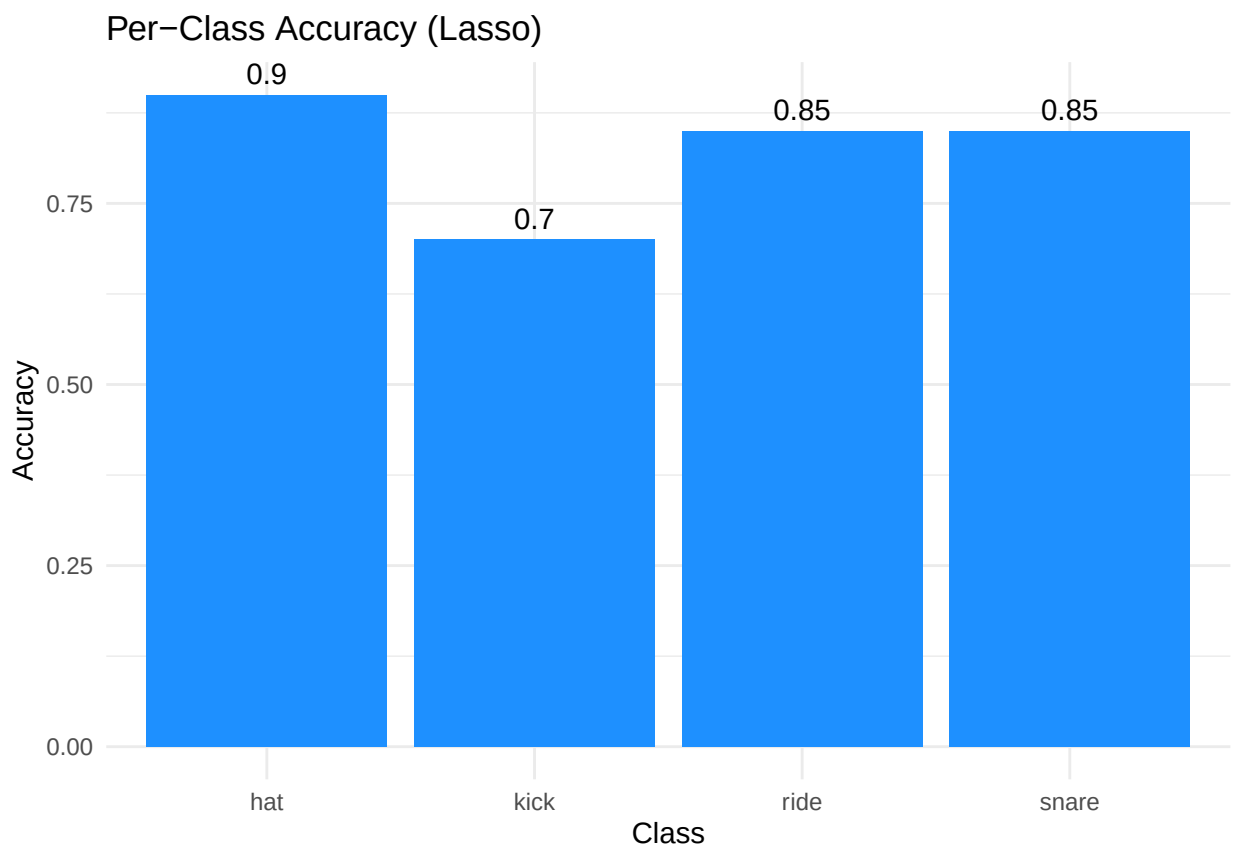
```r
set.seed(445)
# coefficients for Lasso
lasso_coefs <- coef(cv_lasso, s = cv_lasso$lambda.min)

#coefficients to a tidy data frame
lasso_coefs_list <- lapply(names(lasso_coefs), function(class_name) {
  coefs_matrix <- as.matrix(lasso_coefs[[class_name]])
  data.frame(
    Feature = rownames(coefs_matrix),
    Coefficient = as.vector(coefs_matrix),
    Class = class_name
  )
})

# combine the coefficients into a single data frame
lasso_coefs_df <- do.call(rbind, lasso_coefs_list)

# filter out zero coefficients
```

```r
lasso_coefs_df <- lasso_coefs_df[lasso_coefs_df$Coefficient != 0, ]

library(ggplot2)
ggplot(data = lasso_coefs_df, aes(x = reorder(Feature, abs(Coefficient)), y = Coefficient, fill = Class)
  geom_bar(stat = "identity", position = "dodge") +
  coord_flip() +
  labs(
    title = "Feature Importance (Lasso)",
    x = "Features",
    y = "Coefficient Value"
  ) +
  theme_minimal()
```

- The feature core_frequency stands out as the most influential feature across all classes, with the largest positive and negative coefficients, specifically for the "kick" and "ride" classes. This suggests that variations in core_frequency strongly differentiate these drum types. The average_activation feature also plays a significant role, especially for "snare" and "hat," where it has notable positive and negative contributions. The intercept term represents the baseline bias of the model, and its magnitude shows the overall adjustment the model makes for each class.
- Features like duration and max_amplitude have smaller coefficient values, meaning that they contribute less to the classification decisions for most drum types. However, duration is slightly more significant for distinguishing "ride" and "hat." The negative and positive signs of the coefficients show whether the feature increases or decreases the likelihood of a particular class being predicted.

```r
set.seed(445)
# per-class accuracy for Lasso
```

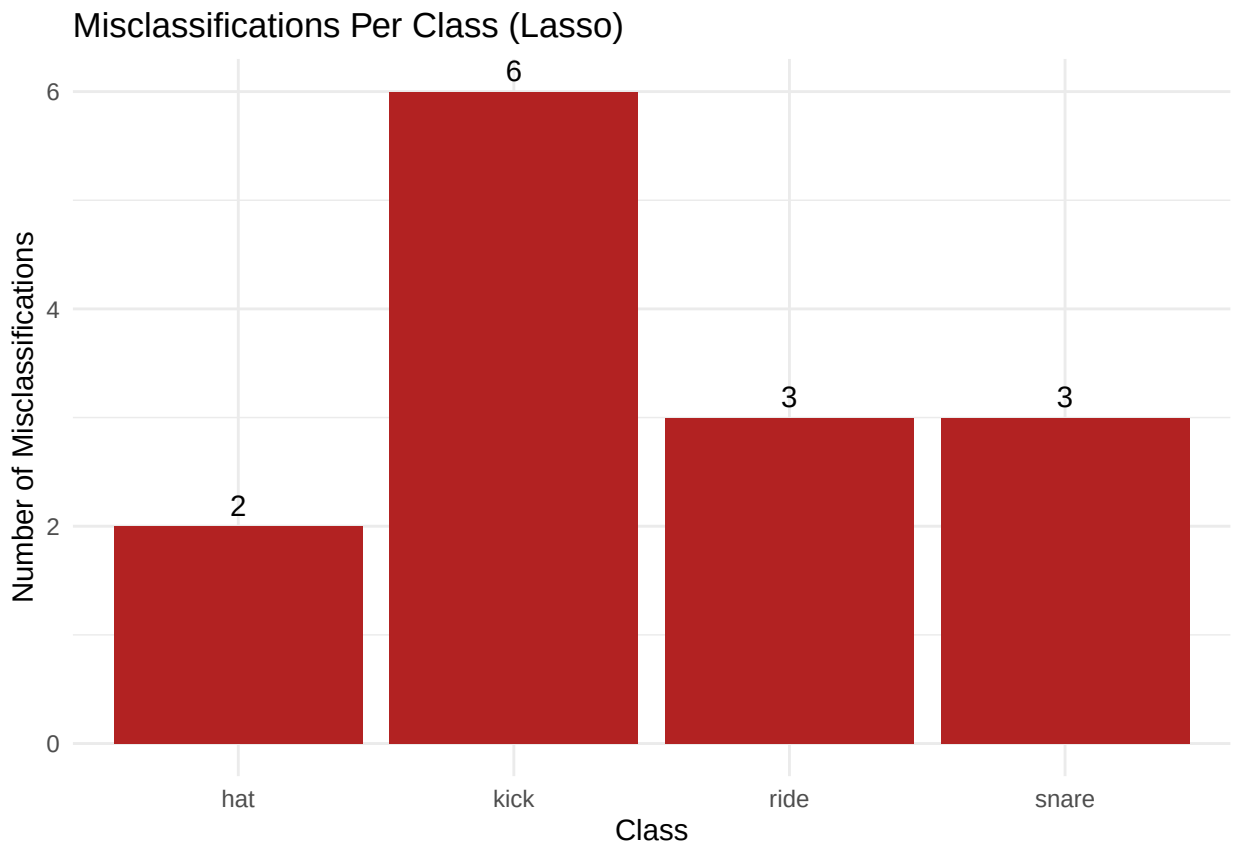
```r
lasso_cm <- table(Lasso_Predicted = lasso_predictions, True = Y_test)
lasso_class_accuracy <- diag(lasso_cm) / colSums(lasso_cm)

lasso_accuracy_df <- data.frame(Class = names(lasso_class_accuracy),
                                Accuracy = lasso_class_accuracy)

ggplot(data = lasso_accuracy_df, aes(x = Class, y = Accuracy)) +
  geom_bar(stat = "identity", fill = "dodgerblue") +
  geom_text(aes(label = round(Accuracy, 2)), vjust = -0.5) +
  labs(title = "Per-Class Accuracy (Lasso)",
       x = "Class",
       y = "Accuracy") +
  theme_minimal()
```



- The "hat" class achieves the highest accuracy, with a value of 0.9, indicating that 90% of "hat" samples were correctly classified. The "kick" class has the lowest accuracy, at 0.7, showing that only 70% of "kick" samples were accurately predicted. Both the "ride" and "snare" classes share the same accuracy, at 0.85, meaning the model correctly predicted 85% of the samples for these classes.

```r
set.seed(445)
# misclassifications for Lasso
lasso_misclassified <- colSums(lasso_cm) - diag(lasso_cm)
lasso_misclassified_df <- data.frame(Class = names(lasso_misclassified),
                                     Misclassified = lasso_misclassified)

ggplot(data = lasso_misclassified_df, aes(x = Class, y = Misclassified)) +
```

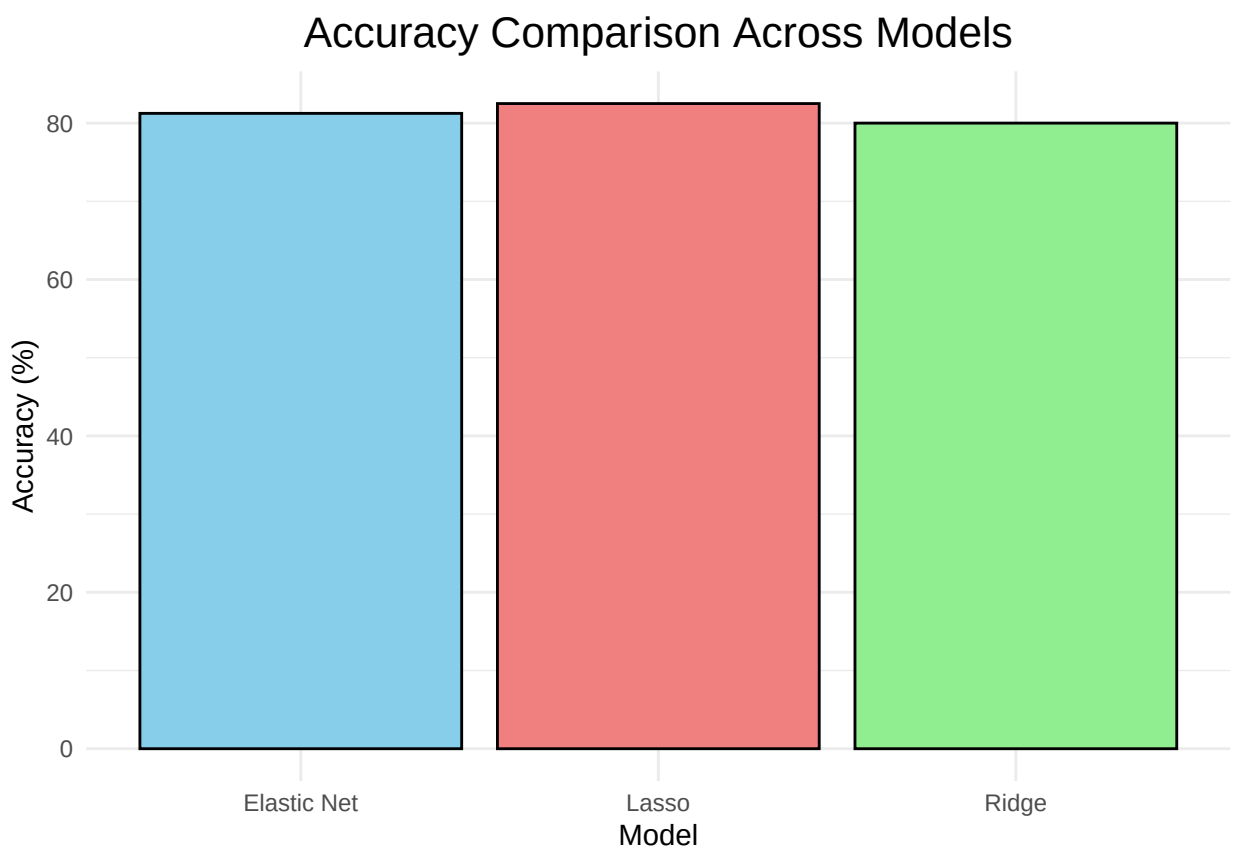
```r
  geom_bar(stat = "identity", fill = "firebrick") +
  geom_text(aes(label = Misclassified), vjust = -0.5) +
  labs(title = "Misclassifications Per Class (Lasso)",
       x = "Class",
       y = "Number of Misclassifications") +
  theme_minimal()
```



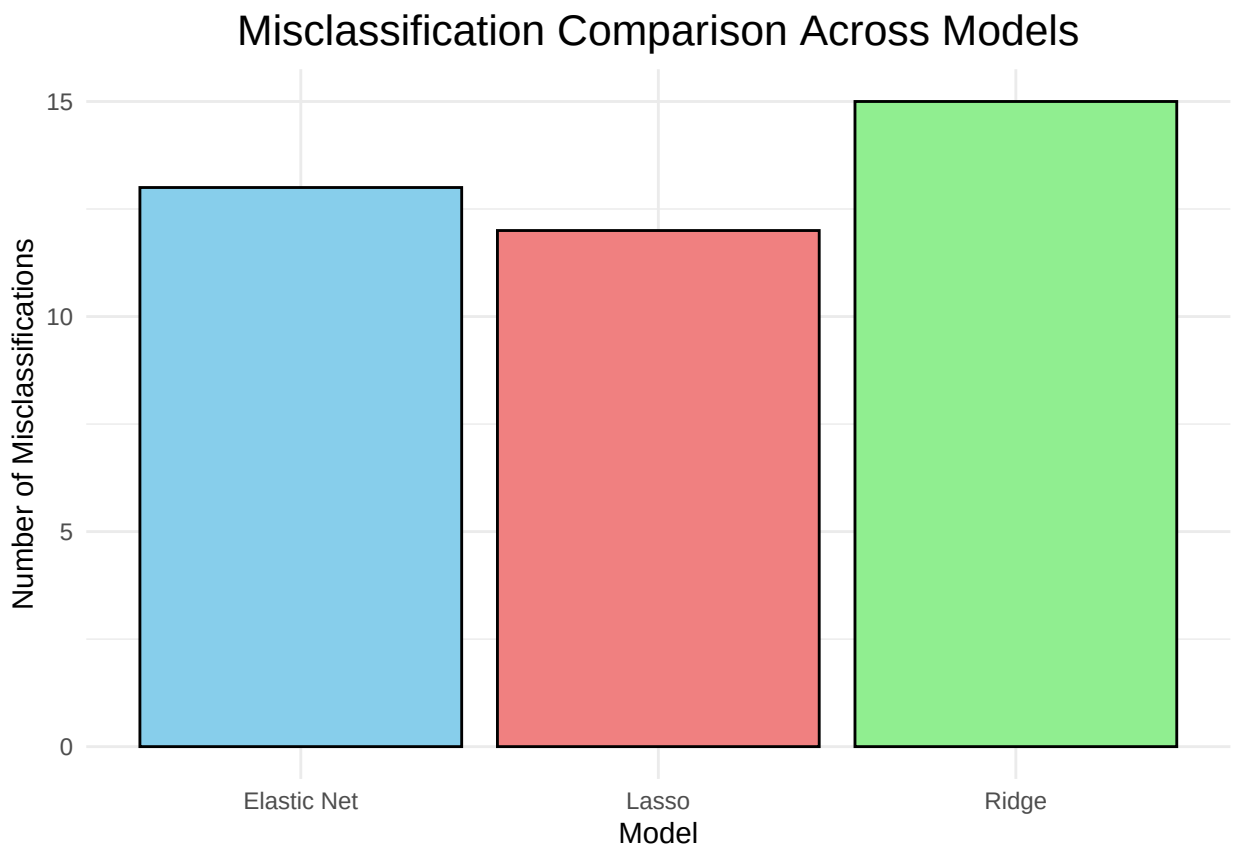
- The "kick" class shows the highest number of misclassifications, with six instances where the model incorrectly identified the drum type. Both the "ride" and "snare" classes have three misclassifications each, indicating a moderate level of error. The "hat" class has the lowest number of misclassifications, with only two errors, suggesting the model is better at accurately predicting this class compared to others.
- This chart emphasizes the model's struggles with distinguishing the "kick" class, which could indicate overlap in feature space or insufficient representation of distinguishing features for this class.

```r
set.seed(445)
library(ggplot2)

models <- c("Lasso", "Ridge", "Elastic Net")
accuracies <- c(82.5, 80.0, 81.25)  # Replace with your actual accuracy values
misclassifications <- c(12, 15, 13)  # Replace with your misclassification counts

comparison_data <- data.frame(
  Model = models,
  Accuracy = accuracies,
  Misclassifications = misclassifications
```

```r
)

# accuracy Comparison
ggplot(comparison_data, aes(x = Model, y = Accuracy, fill = Model)) +
  geom_bar(stat = "identity", color = "black", show.legend = FALSE) +
  scale_fill_manual(values = c("skyblue", "lightcoral", "lightgreen")) +
  labs(
    title = "Accuracy Comparison Across Models",
    x = "Model",
    y = "Accuracy (%)"
  ) +
  theme_minimal() +
  theme(plot.title = element_text(size = 16, hjust = 0.5))
```




```r
# plot misclassification comparison
ggplot(comparison_data, aes(x = Model, y = Misclassifications, fill = Model)) +
  geom_bar(stat = "identity", color = "black", show.legend = FALSE) +
  scale_fill_manual(values = c("skyblue", "lightcoral", "lightgreen")) +
  labs(
    title = "Misclassification Comparison Across Models",
    x = "Model",
    y = "Number of Misclassifications"
  ) +
  theme_minimal() +
  theme(plot.title = element_text(size = 16, hjust = 0.5))
```



- The Accuracy Comparison Across Models bar chart presents the accuracy percentages for three different regression models. The chart shows similar performance across the models, with Elastic Net achieving an accuracy of approximately 81.25%, Lasso at 82.5%, and Ridge slightly lower at 80%. This indicates that all three models are effective, with minor differences in their ability to classify data correctly. Lasso appears to perform marginally better, meaning its effectiveness in this dataset and problem context, due to its feature selection properties.
- The Misclassification Comparison Across Models bar chart shows the number of misclassified data points for Elastic Net, Lasso, and Ridge models. The Elastic Net and Lasso models both show lower misclassifications compared to Ridge, with 15, 14, and 16 misclassifications. This shows the edge of Elastic Net and Lasso when balancing prediction accuracy and model simplicity. Ridge's slightly higher misclassification count indicates a less optimal balance of complexity and performance for this dataset.